

LAB 00 • ONBOARDING

Setup & Debugging Toolkit

Stand up your own Ghost inside Discord and learn to read what it's doing — so that when the labs start, nobody loses a session to setup friction and everyone can debug an instruction that doesn't land.

DIFFICULTY: ONBOARDING

TIME: 30-45 MIN

TRACK: PRE-FLIGHT (BEFORE LAB 1)

YOU LEAVE WITH: A LIVE GHOST + 4 DEBUGGING MOVES

LEARNING OBJECTIVES

- Provision your personal Ghost agent and confirm it responds in your Discord channel.
- Verify Ghost's **memory persists** across separate messages — proof the agent is really yours.
- Internalize the four debugging moves you'll reuse in every later lab.
- Build the mental model: Ghost is a **persistent operator you direct**, not a chat box that forgets.

Why this lab exists

Every later lab assumes you have a working Ghost and can tell the difference between "Ghost is broken" and "I gave Ghost a bad instruction." Nine times out of ten it's the second one. This session gets everyone to the same starting line and hands you a small toolkit for diagnosing the agent instead of guessing. Treat it as pre-flight: you are not producing a portfolio artifact here — you are making sure the cockpit works.

Prerequisites

- A Discord account and the invite link to the course server (*provided by your instructor*).
- Nothing installed locally — Ghost lives in the cloud. You only need a browser or the Discord app.

Part A • Stand up Ghost on Discord

1

Join the course server

Accept the invite and confirm you land in the **#start-here** channel. Read the pinned message top to bottom before doing anything else — it contains the exact provisioning steps for your cohort.

2

Provision your own Ghost

Follow the pinned onboarding to spin up your personal Ghost agent and link it to your account. Each student gets their *own* Ghost — it is not shared. If provisioning fails, post in **#help** with a screenshot rather than retrying blindly.

3

Find your private channel

You'll be given a dedicated channel where your Ghost listens and replies. This is your workspace for every lab in the course — bookmark it. Everything you build happens here.

4

First contact

Say hello and confirm Ghost responds. Then make it prove it's really running and not a canned greeting:

```
# FIRST CONTACT
```

```
Tell me who you are, what tools you have,  
and what you can and can't do right now.
```

Read the answer carefully — this is your first look at the actual capability envelope you'll be operating inside.

5

Confirm memory persists

This is the single most important check in the whole setup. Tell Ghost one durable fact, then start a *separate* message and see if it remembers.

```
# MESSAGE 1
```

```
Remember this: my current project is called Northwind,  
and I always want examples tied to it.
```

```
# MESSAGE 2 (SEND SEPARATELY)
```

```
What's my current project, and give me a one-line example  
that uses it.
```

If Ghost recalls Northwind, your memory is working and the agent is genuinely yours. If it doesn't, flag it in **#help** before moving on — every later lab depends on this.

IF SETUP GOES WRONG

Ghost doesn't respond at all: confirm you're posting in *your* channel, not a shared one. **It responds but forgets across messages:** memory may not be provisioned — don't proceed, get it fixed. **It responds slowly:** normal on first spin-up; give it a moment before assuming failure.

Part B · The debugging toolkit you'll teach

These five habits turn Ghost from a black box into something you can reason about. You'll lean on them constantly once the labs get harder.

1 · Read the reasoning

Before assuming Ghost is broken, ask it to show its work. Most "wrong" answers are perfectly logical given what Ghost thought you meant.

2 · Inspect memory

Bad output is usually bad or missing *context*, not a bad model. Ask Ghost what it actually knows about the task before you rewrite the whole prompt.

3 · Re-steer, don't restart

When an output is 80% right, correct the last result with a sharper instruction. Starting the task over throws away the 80% that worked and often reintroduces the same problem.

4 · Isolate the step

When a multi-step task fails, don't debug the whole chain at once. Run one step, confirm it, then add the next. The break is almost always at a single handoff.

5 · Check the guardrail

If Ghost refuses or stalls, ask which rule stopped it. Often it's doing *exactly* what a previous instruction told it to do, and the fix is to change that rule — not to fight the symptom.

The four debugging prompts (keep these handy)

```
# 1 · SEE ITS THINKING
```

```
Walk me through what you just did, step by step, and why.
```

```
# 2 · CHECK CONTEXT
```

```
Show me everything you have in memory that's relevant here.
```

```
# 3 · RE-STEER
```

```
That's not it — here's exactly what to change. Keep everything else.
```

```
# 4 · FIND THE BREAK
```

```
Do just step one and stop. Show me the result before continuing.
```

These four cover roughly 90% of what goes wrong across the labs. Students who reach for them reflexively stop treating Ghost as magic and start operating it like a tool.

READY-TO-START CHECKLIST

- ☐ Ghost replies in your own Discord channel.
- ☐ Ghost remembers a fact stated in an earlier, separate message.
- ☐ You've run all four debugging prompts at least once and seen what each returns.
- ☐ You can explain, in one sentence, the difference between "Ghost is broken" and "I gave a bad instruction."

All four checked? You're cleared for Lab 1.

FACILITATOR NOTES

- **Swap in the real details:** replace *#start-here*, the onboarding bot name, and the invite flow with your cohort's actual setup before distributing.
- **Budget for stragglers.** Provisioning is where a cohort desyncs. Have a *#help* channel staffed and a buddy system so early finishers assist.
- **Do the memory check as a group.** It's the one failure that silently breaks later labs — verify it live rather than trusting self-report.
- **Demonstrate one debugging prompt on a deliberately vague request** so students see "read the reasoning" surface a misunderstanding in real time.